

Referencia Tecnica

Sistema de Asistencias Digital con Reconocimiento Facial - Practicas Profesionalizantes III, CENT35

Referencia técnica del proyecto

Qué es este documento. La radiografía completa del sistema: tecnologías, arquitectura, modelo de datos, estructura del código, mapa de rutas y flujos de trabajo. Está pensado para leerse de corrido y entender el proyecto entero sin tener que abrir el código. **Documentos complementarios:** - apuntes-entender-el-proyecto.md -> recorridos de código para estudiar y defender. - adr/ -> el *porqué* de cada decisión arquitectónica. - manuales/ -> cómo usar (administrador) y cómo instalar (técnico). - TECH_DEBT.md -> límites conocidos y su plan.

1. Ficha del proyecto

Nombre	Sistema de Asistencias Digital con Reconocimiento Facial
Contexto	Prácticas Profesionalizantes III -- CENT35, Tierra del Fuego, Argentina
Qué resuelve	Reemplaza el registro de asistencia docente en papel/firma por un flujo automatizado con reconocimiento facial
Escala prevista	200 a 400 docentes por institución, múltiples instituciones
Tipo	Aplicación web multi-tenant, monolito modular
Estado	Primera entrega cerrada (v1.0.0), 6 sprints completados
Versión	0.0.1-SNAPSHOT / tag v1.0.0

Los tres hechos del dominio que condicionan todo el diseño

1. **La cámara está en secretaría, no en el aula.** El sistema no observa la clase: la deduce cruzando la hora del registro con los horarios cargados.
2. **El docente es sujeto pasivo.** No se loguea ni usa la interfaz; solo se posiciona frente a la cámara. Por eso Docente y Usuario son entidades separadas.
3. **Los datos biométricos son datos sensibles** (Ley 25.326 + Resolución AAIP 255/2022). De ahí el consentimiento, el cifrado, el no-guardar-fotos y el borrado físico.

El criterio rector

Ante duda, el sistema no registra. Un falso positivo (confundir a un docente con otro) introduce un dato falso en un registro legal; un falso negativo solo obliga a una carga manual. Por eso el umbral es estricto y la carga manual es parte del diseño, no un parche.

2. Tecnologías

2.1 Stack principal

Capa	Tecnología	Versión	Por qué / RNF
Lenguaje	Java	21 (toolchain)	Restricción del proyecto

Capa	Tecnología	Versión	Por qué / RNF
Framework	Spring Boot	3.5.14	Backend obligatorio (RNF-15)
Build	Gradle	Groovy DSL + wrapper	Gestión de dependencias y ejecución
Base de datos	MariaDB	10.4 (vía XAMPP)	Relacional con multi-tenancy por discriminador (RNF-19)
Driver	<code>mariadb-java-client</code>	runtime	Driver nativo (evita el handshake legacy "5.5.5" de MySQL)
ORM	Hibernate 6 (Spring Data JPA)	starter	Mapeo objeto-relacional + filtro multi-tenant
Migraciones	Flyway	core + mysql	Esquema versionado (6 migraciones)
Seguridad	Spring Security	starter	Autenticación, roles, BCrypt, CSRF (RF-01/03, RNF-06/09)
Vistas	Thymeleaf + extras-springsecurity6	starter	Renderizado server-side (RNF-17)
Validación	Bean Validation	starter	Validación declarativa de formularios
AOP	Spring AOP	starter	Aspecto que activa el filtro multi-tenant
Visión por computadora	JavaCV	1.5.11	Wrapper Java de OpenCV (RNF-16)
	OpenCV	4.10.0-1.5.11	Detección (Haar) + reconocimiento (LBPH, módulo contrib)
Utilidades	Lombok	compileOnly	Reduce boilerplate (getters, builders, constructores)
Monitoreo	Actuator	starter	<code>/actuator/health</code> e <code>/info</code>
Dev	DevTools	developmentOnly	Recarga en caliente

2.2 Testing

Herramienta	Uso
JUnit 5	Motor de tests
Mockito	Mocks de repositorios y servicios
AssertJ	Aserciones fluidas
H2	Base en memoria para el perfil <code>test</code>
spring-security-test	Utilidades de seguridad en tests

107 tests en 16 clases, todos verdes.

2.3 Frontend (sin framework)

No hay React/Angular/Vue: **Thymeleaf + JavaScript vanilla**. 8 archivos JS, cada uno con una responsabilidad:

Archivo	Qué hace
<code>pase-asistencia.js</code>	Cámara + loop de reconocimiento + overlay de recuadros
<code>registro-facial.js</code>	Grabación de 30 s para registrar un rostro
<code>toast.js</code>	Notificaciones flotantes
<code>confirm-modal.js</code>	Modales de confirmación declarativos (<code>data-confirm</code>)
<code>navbar.js</code>	Menú responsive con detección de overflow
<code>password-ui.js</code>	Mostrar/ocultar contraseña, bloqueo de copiar/pegar
<code>table-scroll.js</code>	Scroll horizontal con Shift+rueda en tablas anchas

Cámara: API `MediaDevices.getUserMedia()` del navegador (RF-15, RNF-20). Funciona en `localhost` sin HTTPS; en red requiere HTTPS.

2.4 Restricción de plataforma nativa

`gradle.properties` fija `javacpp.platform=windows-x86_64` para que Gradle descargue solo los binarios de OpenCV para Windows (~150 MB en lugar de ~1 GB). En Linux hay que sobrescribir con `-Pjavacpp.platform=linux-x86_64`.

3. Arquitectura

3.1 Vista general



3.2 Las capas y su responsabilidad

Capa	Responsabilidad	Qué NO hace
<code>controller/</code>	Recibe HTTP, valida forma (Bean Validation), delega, elige vista o devuelve JSON	Lógica de negocio
<code>service/</code>	Reglas de negocio, transacciones (@Transactional), validación multi-tenant	Saber de HTTP
<code>repository/</code>	Acceso a datos (Spring Data JPA)	Reglas de negocio
<code>model/</code>	Entidades JPA y enums	Lógica de aplicación
<code>dto/</code>	Transporte UI <-> controller	Persistirse
<code>config/</code>	Seguridad, tenant, scheduling, web/JPA	--

Regla de oro: una operación siempre viaja `controller -> service -> repository -> model`. Las entidades no salen a la vista: para eso están los DTOs.

3.3 Multi-tenancy: el mecanismo en tres capas

Es lo más importante de la arquitectura. Una sola instancia sirve a todas las instituciones con aislamiento total (RF-04, RNF-05, RNF-10).

Capa 1 -- Filtro Hibernate automático

- `model/BaseTenantEntity` aporta la columna `institucion_id`.
- `model/package-info.java` declara el `@FilterDef("tenant")` global.
- Las entidades filtradas llevan `@Filter(name="tenant")`: `**Usuario`, `Docente`, `Carrera`, `Materia`, `Asistencia`**.

- `config/TenantInterceptor` carga el tenant en `TenantContext` (`ThreadLocal`) al inicio de cada request y **lo limpia al final**.

- `config/TenantFilterAspect` activa el filtro en cada `@Service`, agregando automáticamente `WHERE institucion_id = :institucionId`.

Capa 2 -- WHERE explícito en JOINS El filtro de Hibernate **no se propaga a los JOIN de JPQL** (lección TD-003 / ADR-0004). Por eso toda `@Query` con JOIN lleva `WHERE ... institucionId = :tenantId` a mano.

Capa 3 -- Validación en servicios Métodos `obtenerXValidado(id, tenantId)` que verifican pertenencia antes de operar. Ante un acceso cruzado responden **"no encontrado"** (no "prohibido"), para no revelar que el registro existe en otra institución.

Dos incidentes reales documentados (valen como experiencia, no como demérito): **TD-003** -- el filtro no se propagaba a JOINS y un tenant veía comisiones de otro. **TD-007** -- al reorganizar los paquetes, el pointcut del aspecto dejó de coincidir y el filtro quedó *silenciosamente inactivo*. Se corrigió cambiando el pointcut a uno basado en anotación (`@within(@Service)`) y se agregó un test que se pone rojo si vuelve a pasar.

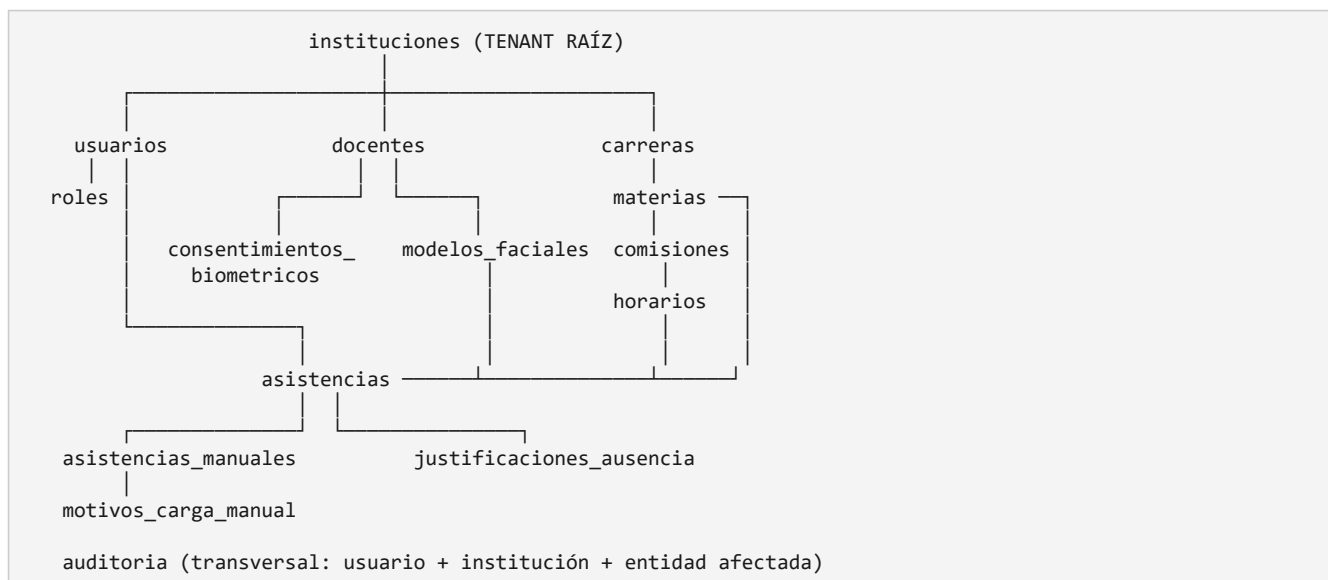
3.4 Decisiones arquitectónicas (ADRs)

ADR	Decisión	Estado
0001	Monolito modular por dominio	Reemplazada por 0006
0002	Multi-tenancy por discriminador (<code>institucion_id</code>)	Vigente
0003	Sesión con cookie HTTP (no JWT)	Vigente
0004	WHERE explícito en JOINS	Vigente
0005	Diseño del consentimiento biométrico	Vigente
0006	Reorganización a package-by-layer	Vigente
0007	Reconocimiento con JavaCV + LBPH (+ desviación del RF-08)	Vigente
0008	Modelo de asistencia automática (+ desempate RF-18)	Vigente

4. Modelo de base de datos

15 tablas, 26 claves foráneas, 15 índices, 11 constraints CHECK. Motor InnoDB, charset `utf8mb4_unicode_ci`.

4.1 Mapa de relaciones



4.2 Las tablas, agrupadas

Núcleo del tenant y acceso

Tabla	Qué guarda	Claves
instituciones	La institución educativa (el tenant)	UNIQUE nombre, UNIQUE cuit
roles	Catálogo global: INSTITUCION, ADMIN	UNIQUE codigo
usuarios	Quien se loguea. password_hash BCrypt	UNIQUE (institucion, username), UNIQUE (institucion, email)

Docentes y biometría

Tabla	Qué guarda	Detalle
docentes	Personal docente (sin login)	UNIQUE (institucion, dni) y (institucion, legajo)
consentimientos_biometricos	Histórico de consentimientos	version_terminos, metodo (ESCRITO/DIGITAL), fechas, vigente, + auditoría forense (IP y user-agent de otorgamiento y revocación, motivo, quién revocó)
modelos_faciales	El dato biométrico	embedding_cifrado LONGBLOB (modelo LBPH gzip + AES), algoritmo, dimensiones, activo, fecha_baja

Estructura académica

Tabla	Qué guarda	Detalle
carreras	Programas académicos	UNIQUE (institucion, codigo)
materias	Asignaturas	FK carrera + FK docente_titular_id (nullable)
comisiones	Divisiones de una materia	FK materia + FK docente_asignado_id (nullable desde V004), CHECK cupo > 0
horarios	Franjas semanales	dia_semana (1-7 ISO), hora_inicio, hora_fin, tolerancia_min (default 15), vigencia. CHECK hora_fin > hora_inicio

Asistencia

Tabla	Qué guarda	Detalle
asistencias	El núcleo del negocio	fecha, hora_registrada, estado (PRESENTE/TARDE/AUSENTE), metodo (AUTOMATICO/MANUAL), modelo_facial_id (FK ON DELETE SET NULL), confianza DECIMAL(5,4). UNIQUE (docente_id, horario_id, fecha) <- garantiza idempotencia
motivos_carga_manual	Catálogo: FALLA_CAMARA, FALLA_RECONOCIMIENTO, NO_REGISTRADO, OTRO	Seed en V001
asistencias_manuales	Detalle 1:1 de una carga manual	Quién la cargó, motivo, detalle libre
justificaciones_ausencia	Detalle 1:1 de una ausencia justificada	Motivo (texto), URL de documento opcional
auditoria	Log de acciones administrativas	valores_anteriores/valores_nuevos en JSON. <i>Tabla creada; registro automático fuera de alcance</i>

4.3 Detalles de diseño que conviene conocer

- **institucion_id denormalizado** en materias y asistencias: refuerza el aislamiento y acelera los reportes, aunque se podría derivar por JOIN.
- **asistencias.modelo_facial_id es ON DELETE SET NULL**: permite el borrado físico del dato biométrico (ARCO) conservando el historial administrativo.
- **UNIQUE (docente_id, horario_id, fecha)**: la garantía de idempotencia a nivel motor. Resuelve la carrera entre el pase facial y el job de ausencias.
- **CHECK ck_asistencias_metodo_modelo**: si el método es MANUAL, no puede haber modelo facial ni confianza.
- **Baja lógica generalizada (activo)** en todas las entidades de negocio, **con una excepción**: el vector biométrico se borra físicamente ante supresión ARCO.

4.4 Migraciones (Flyway)

Versión	Qué hace
V001__init	Esquema completo: 15 tablas con FKs, índices y CHECKs + seed de catálogos
V002__seed_test_data	Datos de prueba (instituciones, usuarios)
V003__rename_rol_superadmin_to_institucion	SUPERADMIN_INSTITUCION -> INSTITUCION
V004__comisiones_docente_nullable	docente_asignado_id pasa a nullable
V005__consentimientos_biometricos_audit	Columnas de auditoría forense (IP, user-agent, motivo, timestamps)
V006__modelos_faciales_mediumblob	embedding_cifrado: BLOB -> LONGBLOB

Regla: una migración aplicada nunca se edita. Todo cambio va en una nueva V00X.

5. Estructura del código

5.1 Árbol del proyecto

```

asistencias/
├── build.gradle                Dependencias y build
├── gradle.properties          javacpp.platform (binarios OpenCV)
├── docs/
│   ├── adr/                   8 decisiones arquitectónicas
│   ├── manuales/              Manual de administrador + técnico
│   ├── uml/                   3 diagramas PlantUML
│   ├── apuntes-entender-el-proyecto.md
│   ├── calibracion-umbral.md   Protocolo de calibración
│   ├── guion-video-demo.md
│   ├── referencia-tecnica.md   (este archivo)
│   ├── TECH_DEBT.md           7 deudas documentadas
│   └── CORRECCIONES.md        Bitácora de pedidos del cliente
├── src/main/
│   ├── java/edu/cent35/asistencias/
│   │   ├── AsistenciasApplication.java
│   │   ├── config/            (9) Seguridad, tenant, scheduling
│   │   ├── controller/        (15) Endpoints web y JSON
│   │   ├── service/           (19) Lógica de negocio
│   │   ├── repository/        (14) Acceso a datos
│   │   ├── model/             (22) Entidades JPA + enums
│   │   └── dto/               (29) Transporte
│   ├── resources/
│   │   ├── db/migration/      (6) Scripts Flyway
│   │   ├── opencv/            haarcascade_frontalface_default.xml
│   │   ├── static/            8 JS + main.css
│   │   ├── templates/         (28) Vistas Thymeleaf
│   │   └── application*.properties
│   └── src/test/java/         (16) 107 tests

```

5.2 Qué hay en cada paquete

config/ -- lo transversal

Clase	Rol
SecurityConfig	Form login, rutas públicas, BCrypt, logout, @EnableMethodSecurity
CustomUserDetails	Principal de Spring Security extendido con institucionId
CustomUserDetailsService	Carga el usuario desde la BD
TenantContext	ThreadLocal con el tenant activo
TenantInterceptor	Lo llena/limpia por request; escribe MDC para los logs
TenantFilterAspect	Activa el filtro Hibernate en cada @Service
PlanificacionConfig	@EnableScheduling (job de ausencias)
WebMvcConfig, JpaConfig	Registro del interceptor, config JPA

service/ -- la lógica de negocio

CRUD y gestión: MiInstitucionService, UsuarioService, DocenteService, CarreraService, MateriaService, ComisionService, HorarioService, GrillaService.

Biometría y visión:

Clase	Responsabilidad
DeteccionRostroService	Haar Cascade: detecta y recorta rostros (gris, 200x200)
MotorLbphService	Entrena/deserializa el modelo LBPH + gzip
CifradoBiometricoService	AES-256-GCM sobre el modelo
ModeloFacialService	Orquesta registro, re-registro y supresión ARCO
IdentificacionFacialService	Identifica un rostro (con cache de recognizers)
ConsentimientoBiometricoService	Otorgar/revocar/estado del consentimiento

Clase	Responsabilidad
TextoConsentimiento	Texto legal versionado (2026-05-v1)

Asistencia:

Clase	Responsabilidad
AsistenciaService	Marca automática y manual, justificación, listado con AUSENTES, desempate RF-18
PaseAsistenciaService	Fachada: identificación + marcado
GeneradorAusenciasService	Job @Scheduled que materializa las ausencias
ReporteAsistenciaService	Reporte filtrado con detalle de manual y justificación

5.3 Convenciones de código

- **Español** en identificadores y comentarios (RegistroAsistencia, no AttendanceRecord).
- **Lombok**: @Getter/@Setter, @Builder, @RequiredArgsConstructor, @Slf4j.
- **Inyección por constructor** (vía @RequiredArgsConstructor), campos final.
- **DTOs como** record cuando son de solo lectura; clases con @Data cuando son formularios con validación.
- **Javadoc con el porqué**, no con el qué. Muchas clases citan el RF/RNF que cubren.
- **Commits**: Conventional Commits (feat, fix, docs, refactor, build, chore).

6. Mapa de rutas

Módulo	Ruta base	Operaciones	Rol requerido
Home / Login	/, /login	GET	público / autenticado
Mi Institución	/mi-institucion	GET, POST	INSTITUCION
Usuarios	/usuarios	listar, nuevo, editar, password	INSTITUCION
Carreras	/carreras	listar, nueva, editar, baja, alta	INSTITUCION o ADMIN
Materias	/materias	listar, nueva, editar, baja, alta	INSTITUCION o ADMIN
Comisiones	/comisiones	listar, nueva, editar, baja, alta	INSTITUCION o ADMIN
Horarios	/horarios	listar, nuevo, editar, baja, alta	INSTITUCION o ADMIN
Grilla	/grilla	GET (vista semanal)	INSTITUCION o ADMIN
Docentes	/docentes	listar, nuevo, editar, baja, alta	INSTITUCION o ADMIN
Consentimiento	/docentes/{id}/consentimiento	otorgar, revocar	INSTITUCION o ADMIN
Rostro	/docentes/{id}/rostro	registrar, suprimir (ARCO)	INSTITUCION o ADMIN
Detección	/reconocimiento/detectar	POST (JSON)	INSTITUCION o ADMIN
Pase	/asistencia/pase	GET, POST /marcar (JSON)	INSTITUCION o ADMIN
Asistencias	/asistencias	listar, manual/nueva, {id}/justificar	INSTITUCION o ADMIN
Reportes	/reportes	GET, GET /csv	INSTITUCION o ADMIN

Patrón REST-ish uniforme: GET /x (listar) - GET /x/nueva + POST /x/nueva (crear) - GET /x/{id}/editar + POST /x/{id}/editar (editar) - POST /x/{id}/baja y /alta (baja lógica).

7. Flujos de trabajo

7.1 Flujo de configuración inicial (el orden importa)

1. Institución — crea Usuarios administradores
2. Admin — crea Carreras
3. — crea Materias (asigna carrera + docente titular)
4. — crea Docentes
5. — crea Comisiones (asigna materia + docente)
6. — crea Horarios (día, horas, TOLERANCIA)
7. — carga CONSENTIMIENTO del docente <- requisito legal
8. — registra el ROSTRO del docente <- requiere el paso 7

El paso 7 bloquea el 8: sin consentimiento vigente, la pantalla de registro facial no permite grabar. Es la regla legal materializada en código.

7.2 Flujo diario de asistencia

```
Docente frente a la cámara
|
[Navegador] captura 1 frame/seg — POST /asistencia/pase/marcar
|
[DeteccionRostroService] ¿hay un rostro claro y único?
| no -> "no se detecta rostro"
| sí
[IdentificacionFacialService] compara contra los modelos del tenant
|
| distancia > umbral — ROJO "no reconocido" — CARGA MANUAL
| distancia <= umbral
[AsistenciaService.marcarAutomatica]
|
| ningún horario en ventana — AMARILLO "no hay clase ahora"
| varios horarios — DESEMPATE (sin marca -> inicio más cercano -> menor id)
| ya existe marca — AZUL "ya estaba marcado" (idempotencia)
|
¿llegó antes o después de hora_inicio?
| antes (dentro de tolerancia) — PRESENTE
| después — TARDE
|
VERDE + guarda en `asistencias` + pausa 5 s
```

7.3 Flujo de cierre del día

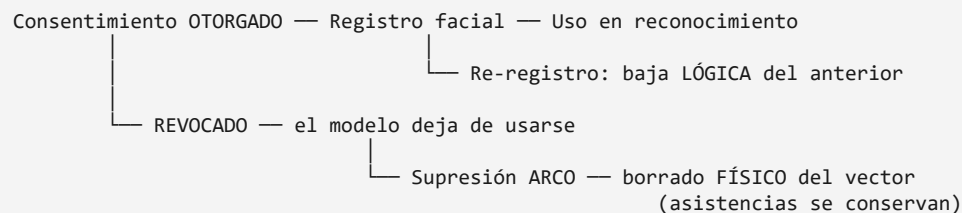
```
[Job @Scheduled cada 30 min]
por cada institución activa:
  setea TenantContext manualmente (no hay request)
  busca horarios del día ya terminados
  sin marca del docente asignado -> crea fila AUSENTE
  (si hay carrera con el pase, el UNIQUE la resuelve)
  limpia TenantContext
```

Mientras tanto, el **listado** muestra las ausencias calculadas al vuelo como ventana de gracia hasta que el job las persista (modelo híbrido, ADR-0008).

7.4 Flujo de excepciones

```
Reconocimiento falló — Carga manual (motivo del catálogo + detalle)
|
Ausencia persistida — Justificación (motivo + documento opcional)
```

7.5 Flujo legal (ciclo de vida del dato biométrico)



7.6 Flujo de desarrollo (cómo se construyó)

Desarrollo incremental por prototipos (RNF-25), 6 sprints + una reorganización:

Sprint	Entregable	Tag
S0	Setup + esquema BD + login dummy	sprint-0-cierre
S1	Multi-tenancy + autenticación real + CRUDs	sprint-1-cierre
S2	CRUD académico + grilla semanal	sprint-2-cierre
S3	Docentes + consentimiento biométrico	sprint-3-cierre
S3.5	Reorganización a package-by-layer	--
S4	Reconocimiento facial (OpenCV/LBPH)	sprint-4-cierre
S5	Asistencia automática end-to-end	sprint-5-cierre
S6	Reportes + UML + manuales + cierre	sprint-6-cierre / v1.0.0

Ciclo de cada fase: implementar -> compilar -> tests -> prueba manual del usuario -> commit con Conventional Commits -> siguiente fase. Cada decisión relevante se documenta en un ADR; cada límite conocido, en `TECH_DEBT.md`.

8. Configuración

8.1 Perfiles

Perfil	Archivo	Uso
(base)	<code>application.properties</code>	Config común, versionada, sin secretos
local	<code>application-local.properties</code>	Credenciales de la BD local. NO versionado
test	<code>application-test.properties</code>	H2 en memoria, Flyway off, job de ausencias off

`./gradlew bootRun` activa `local` automáticamente.

8.2 Propiedades propias del sistema

Propiedad	Default	Para qué
<code>app.biometria.clave-cifrado</code>	<code>env BIOMETRIA_CLAVE</code>	Clave AES del vector biométrico
<code>app.biometria.salt</code>	<code>env BIOMETRIA_SALT</code>	Salt PBKDF2
<code>app.biometria.duracion-grabacion-seg</code>	30	Duración de la grabación de registro
<code>app.biometria.intervalo-captura-ms</code>	1500	Intervalo entre frames (~=20 frames)
<code>app.biometria.minimo-capturas-validas</code>	5	Mínimo de frames con rostro para entrenar
<code>app.biometria.tamano-rostro</code>	200	Lado en px del rostro normalizado

Propiedad	Default	Para qué
app.biometria.umbra1-confianza	100.0	Distancia LBPH máxima para reconocer
app.asistencia.ausencias-habilitado	true	Enciende/apaga el job
app.asistencia.ausencias-cron	0 /30 *	Frecuencia del job

8.3 Ajustes de infraestructura que hubo que hacer

Ajuste	Por qué
spring.jpa.open-in-view=false	Expone los N+1 en vez de ocultarlos. Obliga a JOIN FETCH o a "tocar" los LAZY en el service
server.tomcat.max-http-form-post-size=20MB	Los ~20 frames base64 del registro pesan 2-3 MB
max_allowed_packet=64M en MariaDB	El modelo LBPH cifrado supera el default de 1 MB (causó una corrupción de tablas del sistema durante el desarrollo)
hibernate.jdbc.time_zone=America/Argentina/Ushuaia	Coherencia horaria en las marcas

9. Cómo correr el proyecto

```
# 1. Prerrequisitos: JDK 21, XAMPP con MariaDB corriendo

# 2. Base de datos (en phpMyAdmin como root)
CREATE DATABASE asistenciaautomatica CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'asistencias'@'localhost' IDENTIFIED BY 'TU_PASSWORD';
GRANT ALL PRIVILEGES ON asistenciaautomatica.* TO 'asistencias'@'localhost';
FLUSH PRIVILEGES;

# 3. Crear src/main/resources/application-local.properties con esas credenciales
# (ver manual técnico, sección 3.3)

# 4. Subir max_allowed_packet=64M en C:\xampp\mysql\bin\my.ini y reiniciar MySQL

# 5. Levantar (Flyway aplica V001..V006 solo)
./gradlew bootRun          # -> http://localhost:8080

# Tests
./gradlew test              # 107 tests
```

10. Estado y límites conocidos

Cobertura de requerimientos

De los **37 RF**: la gran mayoría implementados. Pendientes o parciales:

RF/RNF	Estado
RF-08	LBPH en lugar de embeddings -- desviación formalizada (ADR-0007) con migración verificada
RF-31/32/33	Export PDF, Excel y gráficos -> hay CSV
RF-34/35/36	Auditoría: tabla creada, registro automático fuera de alcance
RF-37	Dashboard con métricas del día: parcial
RNF-22	Modo claro conmutable: solo oscuro

RF/RNF	Estado
(nuevo)	Vivacidad / anti-spoofing: no implementado (candidato RF-38)

Deuda técnica documentada

ID	Tema
TD-001	MariaDB ignora nombres de PK (cosmético)
TD-002	Driver MySQL -> MariaDB (resuelto)
TD-003	El filtro no se propaga a JOINS (mitigado con WHERE explícito)
TD-004	Conversión lineal distancia LBPH -> confianza
TD-005	Cache de modelos sin TTL ni límite
TD-006	Reportes sin paginación
TD-007	Aspecto multi-tenant inactivo tras la reorganización (resuelto)

Lo pendiente de validación empírica

- **Calibración del umbral** (RF-16): el código está instrumentado y el protocolo escrito (`calibracion-umbral.md`), falta la sesión de pruebas con personas reales.
- **Medición del RNF-01** (3 s): instrumentado con logs `RNF01`, falta registrar los números.